

平成 2 1 年度

筑波大学大学院博士課程
システム情報工学研究科
コンピュータサイエンス専攻

博士前期課程（一般入学試験 8 月期）

試験問題 基礎科目（情報基礎）

[注意事項]

1. 試験開始の合図があるまで、問題の中を見てはいけません。
2. 解答用紙の定められた欄に、研究科、専攻、受験番号を記入すること。
3. この問題は全部で 10 ページ（表紙を除く）です。
4. 解答用紙を 2 枚（罫線有り）、下書き用紙（白紙）を 1 枚配布します。
5. 問題は全部で 2 問あります。問題Ⅰの解答を 1 枚目、問題Ⅱの解答を 2 枚目の解答用紙に、必ず分けて記入すること。
6. 解答を記述するにあたって、問題Ⅰの解答、問題Ⅱの解答であることを、必ず明記すること。

平成 2 0 年 8 月 2 1 日

(English version is at Pages 6 and after.)

問題 I の解答は、1枚目の解答用紙に記入すること。

解答用紙には、問題 I の解答であることを明記すること。

問題 I 二分探索木 (binary search tree) は、与えられたキーに対する値を高速に検索するための二分木である。二分探索木では、各節にキーと値が格納される。ある節の左部分木はその節のキーよりも小さいキーだけを含み、右部分木はその節のキーよりも大きいキーだけを含む。この問題では、二分探索木でキーの重複を許さないものとする。図1に、6組のキーと値を持つ二分探索木の例を示す。図1で、各節の上段にキー、下段に値が表示されている。たとえば、キー40の節は、値は、1を保持している。

二分探索木を、図2に示したようなC言語の構造体と大域変数を用いて表現する。変数 `root` は、二分探索木の根を保持しており、初期値は、`NULL` である。`NULL` は、どの要素も指していないことを意味する定数である。以下の設問に答えなさい。

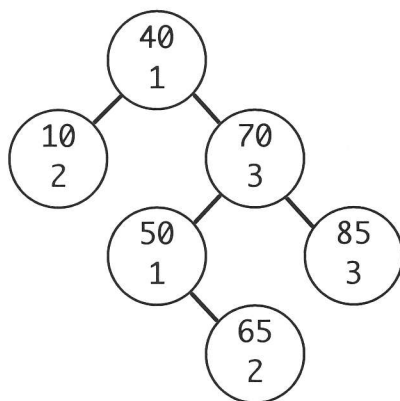


図1 二分探索木の例

```
struct node {
    int key;
    int value;
    struct node *left;
    struct node *right;
};

struct node *root = NULL;
```

図2 二分探索木を表現するC言語の構造体と大域変数

(1) 以下のプログラムで `height()` は、二分探索木の高さを求める関数である。空欄を埋めてプログラムを完成させない。

```
int height1( struct node *p ) {
    int a, b;
    if( p == NULL )
        return 0;
    a = height1( (A) );
    b = height1( (B) );
    if( a >= b )
        return (C) ;
    else
        return (D) ;
}

int height() {
    return height1( root );
}
```

(2) 以下のプログラムで `print()` は、二分探索木に含まれるキーと値を画面に表示する関数である。変数 `root` が図1に示したよう二分探索木を保持していたとする。この関数が呼ばれた時、画面にどのような出力がなされるかを示しなさい。

```
void print1( struct node *p, int level ) {
    int i;
    if( p == NULL )
        return;
    for( i=0; i<level; i++ )
        printf("*");
    printf("key:%d, value:%d\n",p->key,p->value );
    print1( p->left,level+1 );
    print1( p->right,level+1 );
}

void print() {
    print1( root, 1 );
}
```

(3) 以下のプログラムで `get()` は、引数で与えられたキーを持つ節を検索し、その節が持っている値を返す。ただし、そのような節が見つからない時には、`-1` を返す。空欄を埋めてプログラムを完成させない。

```
int get1( struct node *p, int key )
{
    if( p == NULL )
        return -1;
    if(  (A)  )
        return p->value;
    else if(  (B)  )
        return  (C) ;
    else
        return  (D) ;
}

int get( int key ) {
    return get1( root, key );
}
```

(4) 以下のプログラムで `put()` は、二分探索木にキーと値の組を挿入する関数である。
`put()` は、2つの引数、キーと値を取り、そのキーと値を持つ節を二分探索木に挿入する。ただし、既にそのキーを持つ節が含まれていた時には、その節の値を上書きする。空欄を埋めてプログラムを完成させない。ただし、関数 `malloc()` は、指定されたバイト数のメモリを割り当て、その番地を返す関数である。この問題では、メモリの割り当ては常に成功するものとする。

```

struct node *make_node( int key, int value ) {
    struct node *p;
    p = malloc( sizeof(struct node) );
    p->key    = key;
    p->value  = value;
    p->left   = NULL;
    p->right  = NULL;
    return p;
}

void put1( struct node *p, int key, int value ) {
    if(  )
        p->value = value;
    else if(  ) {
        if(  == NULL )
             = make_node( key, value );
        else
            ;
    }
    else {
        if(  == NULL )
             = make_node( key, value );
        else
            ;
    }
}

void put( int key, int value ) {
    if( root == NULL )
        root = make_node( key, value );
    else
        put1( root, key, value );
}

```

(5) 二分探索木は、たとえ同じキーと値の集合を保持していたとしても、データを挿入する順序(関数 `put()` を呼び出す順序)で木の形が異なる。図1と同じキーと値を保持し、木の高さが最も高くなるような二分探索木を作りたい。そのための挿入順序の例を、以下の空欄を埋めて示しなさい。

```
main()
{
    put( 

|     |
|-----|
| (A) |
|-----|

 );
    put( 

|     |
|-----|
| (B) |
|-----|

 );
    put( 

|     |
|-----|
| (C) |
|-----|

 );
    put( 

|     |
|-----|
| (D) |
|-----|

 );
    put( 

|     |
|-----|
| (E) |
|-----|

 );
    put( 

|     |
|-----|
| (F) |
|-----|

 );
    print();
    printf("height: %d\n", height() );
}
```

(6) 二分探索木を用いた探索の効率、木の形だけでなく探索するキーの頻度にも影響される。キーの探索頻度が以下の表1のようになっているとする。探索における比較回数が最小になるような二分探索木の例を1つ求めなさい。その二分探索木を図1と同じ表記方法(節と枝)で解答しなさい。求める二分探索木は、図1のキーと値の組と同じものを持つものとする。

表1 キーの探索頻度

キー	探索頻度
10	70回
85	30回
その他のキー	0回

問題Ⅱの解答は2枚目の解答用紙に記入すること。解答用紙には問題Ⅱの解答であることを明記すること。

問題Ⅱ 以下の設問に答えよ。ただし、設問(2)と設問(3)はブール代数の範疇で答えること。設問(2)と設問(3)においては、特に断らない限り、全ての式は論理式、全ての変数は論理変数、“+”は論理和、“ $\cdot$ ”は論理積、“ $\neg$ ”は論理否定を表す。

(1) 2つの異なる集合 $A = \{a, b\}$ と $B = \{1, 2\}$ を考える。ただし、 a と b は実数であり、 $a \neq b$ である。このとき、以下の問いに答えよ。

(a) $A \times B$ (A と B の直積集合) を求めよ。

(b) A と B の二項関係として $R = \{(x, y) \mid x > y\}$ を考える。 $xRy = \{(a, 1), (a, 2)\}$ のとき、 a と b が満たすべき条件を述べよ。

(2) 次の等式が成り立つことを証明せよ。ただし、真理値表を用いないこと。

(a) $\bar{x} \cdot y + x \cdot \bar{y} + x \cdot y = x + y$

(b) $x \cdot \bar{y} + x \cdot y \cdot z = x \cdot \bar{y} + x \cdot z$

(3) 図に示すように、7セグメントディスプレイに“a”，“b”，“c”，“d”の4個のアルファベットを表示する論理回路を設計する。論理回路への入力は x_1 ， x_2 であり、これらを用いて表示するアルファベットを次のように指定する。

- ・ “a” : $x_1 = 0$ かつ $x_2 = 0$
- ・ “b” : $x_1 = 0$ かつ $x_2 = 1$
- ・ “c” : $x_1 = 1$ かつ $x_2 = 0$
- ・ “d” : $x_1 = 1$ かつ $x_2 = 1$

また、論理回路の出力は z_i ($i = 1, 2, \dots, 7$) であり、

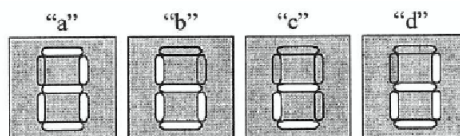
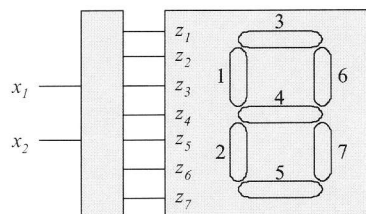
これらを用いて個々のセグメントの状態（点灯、あるいは消灯）を制御する。ここで、図に示すように各セグメントには1から7の番号が付けられており、 $z_i = 1$ のときに限りセグメント i が点灯し、 $z_i = 0$ のときに限りセグメント i が消灯する。このとき、以下の問いに答えよ。

(a) 出力 z_i ($i = 1, 2, \dots, 7$) の真理値表を作成せよ。

(b) 出力 z_6 と z_7 の論理式を積和標準形により示せ。

(c) 出力 z_6 と z_7 のカルノー図を作成し、各々の最簡形の論理式を示せ。

(d) (c) で求めた論理式を正論理 (positive-logic, または active-high) で表す回路図を描け。



白塗りのセグメントは「点灯」、灰塗りのセグメントは「消灯」を表す。

Write the answers of Problem I in the first answer sheet, clearly indicating a heading "Problem I".

Problem I A binary search tree (BST) is a binary tree for the fast retrieval of given keys. In a BST, each node has a key and a value. The left subtree of a node holds keys that are less than the node's key, and the right subtree of a node holds keys that are greater than the node's key. In this problem, we disallow duplicate keys in a BST. Figure 1 shows an example of a BST that holds six key-value pairs. In Figure 1, each node has the key in the first row, and its value in the second row. For example, the node with the key 40 has the value 1.

We implement a BST by using the structure and the global variable in the C language as shown in Figure 2. The variable `root` holds the root node of a BST, and it is initialized with `NULL`. The constant `NULL` means not pointing to anything. Answer the following questions.

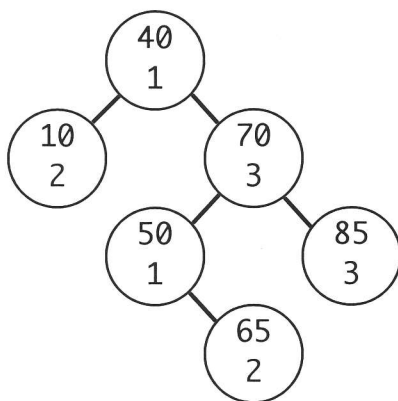


Figure 1 An example of a binary search tree.

```

struct node {
    int key;
    int value;
    struct node *left;
    struct node *right;
};

struct node *root = NULL;
  
```

Figure 2 The structure and the global variable to implement a binary search tree in the C language.

(1) In the following program, `height()` is a function that returns the height of a BST. Fill in the boxes and complete the program.

```

int height1( struct node *p ) {
    int a, b;
    if( p == NULL )
        return 0;
    a = height1(  );
    b = height1(  );
    if( a >= b )
        return  ;
    else
        return  ;
}

int height() {
    return height1( root );
}
  
```

(2) In the following program, `print()` is a function that prints the keys and their values in a BST. Suppose that the variable `root` holds the BST in Figure 1. When this function is called, what is the output of this program to the display?

```
void print1( struct node *p, int level ) {
    int i;
    if( p == NULL )
        return;
    for( i=0; i<level; i++ )
        printf("*");
    printf("key:%d, value:%d\n",p->key,p->value );
    print1( p->left,level+1 );
    print1( p->right,level+1 );
}

void print() {
    print1( root, 1 );
}
```

(3) The following function `get()` searches for a tree node that has the key in the argument, and returns the value of the node. When no such node exists, the function returns `-1`. Fill in the boxes and complete the program.

```
int get1( struct node *p, int key )
{
    if( p == NULL )
        return -1;
    if(  (A)  )
        return p->value;
    else if(  (B)  )
        return  (C) ;
    else
        return  (D) ;
}

int get( int key ) {
    return get1( root, key );
}
```

(4) In the following program, `put()` is a function that inserts a key-value pair into a BST. The function `put()` takes two arguments, a key and its value, and inserts a node that has this key and its value into the BST. When there exists a node that has the same key, this function replaces its value with the given one. Fill in the boxes and complete the program. Note that `malloc()` is a function that allocates a memory block of a specified size (in bytes) and returns its address. In this problem, assume that the memory allocation always ends successfully.

```

struct node *make_node( int key, int value ) {
    struct node *p;
    p = malloc( sizeof(struct node) );
    p->key    = key;
    p->value  = value;
    p->left   = NULL;
    p->right  = NULL;
    return p;
}

void put1( struct node *p, int key, int value ) {
    if(  )
        p->value = value;
    else if(  ) {
        if(  == NULL )
             = make_node( key, value );
        else
            ;
    }
    else {
        if(  == NULL )
             = make_node( key, value );
        else
            ;
    }
}

void put( int key, int value ) {
    if( root == NULL )
        root = make_node( key, value );
    else
        put1( root, key, value );
}

```

(5) In BSTs, the shapes of trees can vary even when these trees have the same set of key-value pairs according to the order of insertion (the order of calling the function `put()`). We are building one of the highest BST that has the same key-value pairs in Figure 1. Answer the order of insertion by filling in the following boxes.

```
main()
{
    put( 

|     |
|-----|
| (A) |
|-----|

 );
    put( 

|     |
|-----|
| (B) |
|-----|

 );
    put( 

|     |
|-----|
| (C) |
|-----|

 );
    put( 

|     |
|-----|
| (D) |
|-----|

 );
    put( 

|     |
|-----|
| (E) |
|-----|

 );
    put( 

|     |
|-----|
| (F) |
|-----|

 );
    print();
    printf("height: %d\n", height() );
}
```

(6) The efficiency of a BST search depends not only on the shapes of the trees but also on the frequencies of the keys used for searching. Suppose that the frequencies of keys used for searching are shown in Table 1. Build a BST that minimizes the number of comparisons in the search. Write the BST using the notation of Figure 1 (with nodes and branches). The BST must include the same key-value pairs as Figure 1.

Table 1 Frequencies of keys used for searching.

Keys	Frequencies
10	70 times
85	30 times
Other keys	0 times

Write the answers of Problem II in the second answer sheet, clearly indicating a heading “Problem II”.

Problem II Answer the following questions. Note that you must answer the questions (2) and (3) in the domain of Boolean algebra. In the questions (2) and (3), if not specially noticed, all expressions, all variables, “+”, “ $\cdot$ ”, and “ $\neg$ ” represent logical expressions, logical variables, logical-OR, logical-AND, and logical-NOT, respectively.

(1) Consider two different sets $A = \{a, b\}$ and $B = \{1, 2\}$, where a and b are real numbers with $a \neq b$. Answer the following questions.

(a) Show $A \times B$, that is, the direct product of A and B .

(b) Consider $R = \{(x, y) \mid x > y\}$ as the binary relation of A and B . When $xRy = \{(a, 1), (a, 2)\}$, find the condition that a and b should satisfy.

(2) Prove that the following equations hold. Do not use any truth tables.

(a) $\bar{x} \cdot y + x \cdot \bar{y} + x \cdot y = x + y$

(b) $x \cdot \bar{y} + x \cdot y \cdot z = x \cdot \bar{y} + x \cdot z$

(3) We design a logical circuit for displaying four alphabets “a”, “b”, “c”, and “d” on a seven-segment display, as shown in the figure. The inputs to the logical circuit are x_1 and x_2 , which specify an alphabet as follows.

- “a”: $x_1 = 0$ and $x_2 = 0$
- “b”: $x_1 = 0$ and $x_2 = 1$
- “c”: $x_1 = 1$ and $x_2 = 0$
- “d”: $x_1 = 1$ and $x_2 = 1$

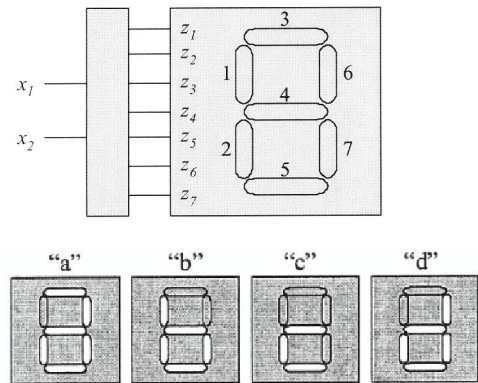
The outputs from the logical circuit are z_i ($i = 1, 2, \dots, 7$), which control the state of each segment (lighting or non-lighting). Each segment is numbered from 1 to 7, and the i th segment is lighting only when $z_i = 1$ and non-lighting only when $z_i = 0$. Answer the following questions.

(a) Show the truth table for the outputs z_i ($i = 1, 2, \dots, 7$).

(b) Describe the logical expressions for each of the outputs z_6 and z_7 in the canonical sum-of-products form.

(c) Show the Karnaugh maps and the simplest logical expressions for each of the outputs z_6 and z_7 .

(d) According to the result of the question (c), draw the logical circuits for each of the outputs z_6 and z_7 . Assume that all signals are expressed in positive-logic (active-high).



In this figure, the white and gray segments mean “lighting” and “non-lighting”, respectively.